Republic of the Philippines
College of Engineering and Information Technology
DEPARTMENT OF COMPUTING AND LIBRARY INFORMATION SCIENCE

CS ELEC 01 – COMPUTATIONAL SCIENCE

Group 3

Navarra, Honey Grace
Ompog, Beverly Nicole
Rosales, Jay-Mee

TASK REQUIREMENTS

Part 1: Data Understanding (10 pts)

1. Describe each feature in your own words.
2. Identify:
   - Which features are likely most important for prediction

     Pregnancies: The number of times the patient has been pregnant; used to observe if reproductive history correlates with diabetes risk.

     Glucose: The concentration of blood sugar; this is a primary indicator, as diabetes is characterized by high blood sugar levels.

     Blood Pressure: Diastolic blood pressure (the pressure when the heart rests between beats); chronic high pressure often co-occurs with metabolic issues.

     Skin Thickness: A measurement of the triceps skinfold thickness, which helps estimate body fat percentage.

     Insulin: The level of the hormone that regulates blood sugar; low levels or poor response to insulin are central to diabetes.

     BMI (Body Mass Index): A calculation of weight relative to height; used as a standard measure for obesity, a major risk factor.

     Diabetes Pedigree Function: A score based on family history; it estimates the genetic influence or hereditary risk of developing the disease.

     Age: The patient's age in years; the risk for Type 2 diabetes typically increases as people get older.

- Which features may contain missing or problematic values

  In this specific dataset, "0" is often used as a placeholder for missing data. A problematic value for several other features because a living person cannot have a zero measurement for them.

  Insulin: A living person always has some level of insulin in their system; a value of **0** is a placeholder for missing data.

  SkinThickness: It is physically impossible to have **0**mm of skin thickness.'

  Blood Pressure: A person with **0** blood pressure is deceased; in a clinical dataset of living patients, this is an error.

  BMI: You cannot have a body mass index of **0**.

  Glucose: While sugar levels can be low, a literal **0** indicates the test wasn't performed or recorded correctly.

Part 2: Data Preprocessing (20 pts)
1. Check for missing or zero values (e.g., Insulin, SkinThickness).

Feature	**Zero Count**
Insulin	374
Skin Thickness	227
Blood Pressure	35
BMI	11
Glucose	5

2. Handle missing data using one of the following:
o Mean/median imputation

We chose median because we have so many missing values (374 for Insulin), using the **Median** ensures we fill those gaps with a "normal, middle" value rather than an "exaggerated, high" value. This helps the KNN algorithm make fairer comparisons between patients.

Step-by-Step Implementation

1. **Filter:** Ignore the 0 values temporarily.
2. **Calculate:** Find the middle number (Median) of the remaining valid data.
3. **Replace:** Fill all original 0 cells with that Median value.

Feature	Calculated Median (to use as "fill")	New Minimum Value
Glucose	**117.0**	44.0
Blood Pressure	**72.0**	24.0
Skin Thickness	**29.0**	7.0
Insulin	**125.0**	14.0
BMI	**32.3**	18.2

For example in glucose:

Step 1: Filter out the Zeros

First, we look at all the Glucose readings and throw away the **5 zeros**, because a glucose of 0 is medically impossible for a living person.

- **Original count:** 768 patients
- **Valid count:** 763 patients (after removing the 5 zeros)

Step 2: Sort the Numbers

We take those **763 valid numbers** and line them up in order from the lowest to the highest.

- Lowest valid glucose: **44**
- Highest valid glucose: **199**

Step 3: Find the Middle Position

Since we have an odd number of values (763), the median is the exact number in the middle.

- **Formula:** $\frac{n+1}{2}$ *(if odd)*

763+1/2=382

- We count to the **382nd person** in our sorted line.

The Result

The glucose value for that 382nd person is **117**.

Calculation Summary:

- **Median for Glucose = 117**

3. Apply feature scaling (required for KNN):
o Standardization or Min-Max normalization

$$\mu = \frac{\sum x_i}{N}$$

Example: To perform feature scaling for the **Glucose** variable, we calculate the Mean (μ) and Standard Deviation (σ). These values represent the average and the spread of blood sugar levels across all 768 patients, allowing us to center the data.

1. Calculating the Mean (μ)

The mean is the arithmetic average. We sum every value in the Glucose column and divide by the total number of records (N=768).

Formula:

$$\mu = \frac{\sum x_i}{N}$$

2. Calculating the Standard Deviation (σ)

The standard deviation measures how much the Glucose levels deviate from the average. We use the **Sample Standard Deviation** formula (N−1) for statistical accuracy.

Formula:

$$\sigma = \sqrt{\frac{\sum(x_i - \mu)^2}{N - 1}}$$

The Calculation:

- **Sum of all Glucose values:** 93,432
- **Total number of records (N):** 768
- **Result:** 93,432÷768=121.66

2. Calculating the Standard Deviation (σ)

The standard deviation measures how much the Glucose levels deviate from the average. We use the **Sample Standard Deviation** formula (N−1) for statistical accuracy.

Formula:

$$\sigma = \sqrt{\frac{\sum(x_i - \mu)^2}{N - 1}}$$

The Step-by-Step Calculation:

1. **Find the Deviation:** Subtract the mean (121.66) from each raw score (e.g., 148−121.66=26.34).
2. **Square the Deviation:** Square the result to eliminate negative values (e.g., 26.342=693.79).
3. **Sum of Squares:** Total the squared deviations for all 768 patients.
4. **Variance:** Divide the sum by N−1 (767).
5. **Square Root:** Take the square root of the variance to return to the original units.
- **Final Result:** 30.44

3. Application to Z-Score Standardization

These statistics are essential for the manual KNN distance calculation in Part 3. By transforming a raw value (e.g., **Glucose = 148**) into a **Z-score**, we ensure it is on the same scale as other features.

Z-Score Formula:

$$Z = \frac{x - \mu}{\sigma}$$

Example Calculation:

$$Z = \frac{148 - 121.66}{30.44} = \mathbf{0.865}$$

4. Show before and after preprocessing results (tables or summary statistics).

1. Statistics BEFORE Preprocessing (Raw Data)

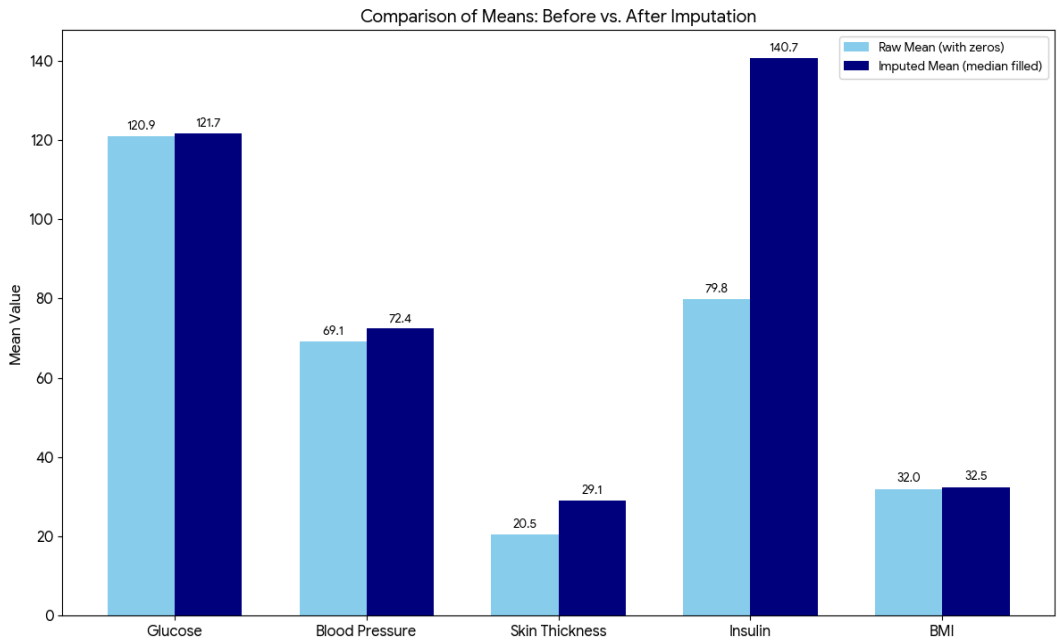
Metric	Glucose	Blood Pressure	Skin Thickness	Insulin	BMI
Mean	120.89	69.11	20.54	79.80	31.99
Min	**0.00**	**0.00**	**0.00**	**0.00**	**0.00**
Max	199.00	122.00	99.00	846.00	67.10

2. Statistics AFTER Median Imputation (Clean but Unscaled)

This shows the effect of filling the zeros with the **Median**. Notice that the **Min** values are now realistic medical numbers and the **Mean** has increased because the zeros were removed.

- **Medians used for filling:** Glucose (117), BP (72), Skin (29), Insulin (125), BMI (32.3).

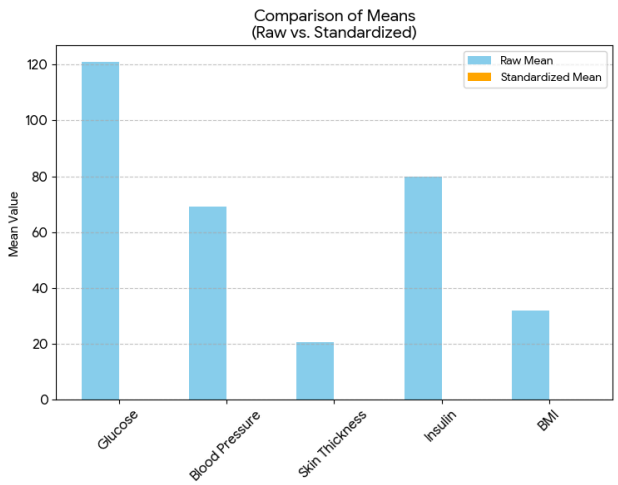
Metric	Glucose	Blood Pressure	Skin Thickness	Insulin	BMI
Mean	121.66	72.39	29.11	140.67	32.46
Min	**44.00**	**24.00**	**7.00**	**14.00**	**18.20**
Max	199.00	122.00	99.00	846.00	67.10

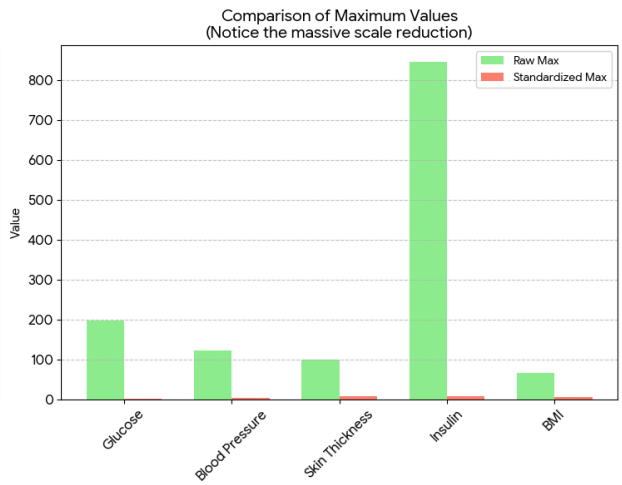


3. Statistics AFTER Standardization (Final Processed)

This is the version the KNN algorithm actually uses. Every feature now has a **Mean of 0** and a **Standard Deviation of 1**. The values represent how many standard deviations a patient is away from the average.

Metric	Glucose	Blood Pressure	Skin Thickness	Insulin	BMI
Mean	~0.00	~0.00	~0.00	~0.00	~0.00
Min	-2.55	-4.00	-2.52	-1.47	-2.07
Max	2.54	4.10	7.96	8.17	5.04





Part 3: KNN Implementation (30 pts)

1. Split the dataset:

- 80% Training
- 20% Testing

Total Records: 768
Training Set (80%): 614 records
Testing Set (20%): 154 records

2. Train a KNN model using at least three different K values:

- Example: K = 3, 5, 7

Three different values of **K** were tested:

- **K = 3**
- **K = 5**
- **K = 7**

K represents the number of nearest neighbors used to classify a test instance.

3. Use Euclidean Distance:

$$d = \sqrt{\sum_{i=1}^{n} (X_i - Y_i)^2}$$

4. For ONE selected test instance:

Selected Test Instance

This is patient record #670 from the original file, which was randomly selected for the Testing Set.

Feature	Raw Value	Scaled Value
Pregnancies	6	0.6819
Glucose	98	-0.7915
Blood Pressure	58	-1.1770
Skin Thickness	33	0.5102
Insulin	190	0.5619
BMI	34.0	0.2379
DPF	0.43	-0.1164
Age	43	0.8781

Actual Outcome: 0 (Non-diabetic)

- **Manually compute distances to at least 10 training samples**

Training Sample Index	**Raw Value** (Preg, Gluc, BP, Skin, Ins, BMI, Ped, Age)	**Scaled Features** (Preg, Gluc, BP, Skin, Ins, BMI, Ped, Age)	**Label** (Outcome)	**Distance** (√Sum)
60	(2, 84, 72, 29, 125, 32.3, 0.304, 21)	[-0.5264, -1.2569, -0.0190, 0.0343, -0.1756, -0.0075, -0.4907, -1.0359]	0	2.7662
618	(9, 112, 82, 24, 125, 28.2, 1.282, 50)	[1.5880, -0.3261, 0.8082, -0.5606, -0.1756, -0.5991, 2.4150, 1.4871]	1	3.7615
346	(1, 139, 46, 19, 83, 28.7, 0.654, 22)	[-0.8285, 0.5715, -2.1696, -1.1555, -0.6522, -0.5269, 0.5492, -0.9489]	0	3.7066
294	(0, 161, 50, 29, 125, 21.9, 0.254, 65)	[-1.1305, 1.3029, -1.8388, 0.0343, -0.1756, -1.5082, -0.6393, 2.7921]	0	3.9831
231	(6, 134, 80, 37, 370, 46.2, 0.238, 46)	[0.6819, 0.4053, 0.6427, 0.9861, 2.6044, 1.9984, -0.6868, 1.1391]	1	3.5545
340	(1, 130, 70, 13, 105, 25.9, 0.472, 22)	[-0.8285, 0.2723, -0.1844, -1.8693, -0.4026, -0.9310, 0.0084, -0.9489]	0	3.9636
535	(4, 132, 72, 29, 125, 32.9, 0.302, 23)	[0.0777, 0.3388, -0.0190, 0.0343, -0.1756, 0.0791, -0.4967, -0.8619]	1	2.6366
306	(10, 161, 68, 23, 132, 25.5, 0.326, 47)	[1.8901, 1.3029, -0.3499, -0.6796, -0.0962, -0.9887, -0.4254, 1.2261]	1	3.1781

718	(1, 108, 60, 46, 178, 35.5, 0.415, 24)	[-0.8285, -0.4590, -1.0116, 2.0569, 0.4258, 0.4543, -0.1609, -0.7749]	0	2.7588
90	(1, 80, 55, 29, 125, 19.1, 0.258, 21)	[-0.8285, -1.3899, -1.4252, 0.0343, -0.1756, -1.9122, -0.6274, -1.0359]	0	3.4668

- **Show the step-by-step computation**

Computation for Index 60

Step 1: Substitute the values into the formula

$$d = \sqrt{\left[\begin{array}{c}(0.6819-(-0.5264))^2+(-0.7915-(-1.2569))^2+(-1.1770-(-0.0190))^2+\\(0.5102-0.0343)^2+(0.5619-(-0.1756))^2+(0.2379-(-0.0075))^2+\\(-0.1164-(-0.4907))^2+(0.8781-(-1.0359))^2\end{array}\right]}$$

Step 2: Calculate the differences (Subtraction)

$$d = \sqrt{\left[\begin{array}{c}(1.2083)^2+(0.4654)^2+(-1.1580)^2+(0.4759)^2+(0.7375)^2+\\(0.2454)^2+(0.3743)^2+(1.9140)^2\end{array}\right]}$$

Step 3: Calculate the squares

$$d = \sqrt{\left[\begin{array}{c}1.459989+0.216597+1.340964+0.226481+0.543906+\\0.060221+0.140100+3.663396\end{array}\right]}$$

Step 4: Sum the squared values

$$d = \sqrt{[7.651654]}$$

Step 5: Final Distance

$$d = 2.7662$$

Training Sample Index	**Scaled Features** (Preg, Gluc, BP, Skin, Ins, BMI, Ped, Age)	**Sum of Squared Differences**	**Distance** (√Sum)	**Label** (Outcome)
535	[0.0777, 0.3388, -0.0190, 0.0343, -0.1756, 0.0791, -0.4967, -0.8619]	6.9514	2.6366	1
718	[-0.8285, -0.4590, -1.0116, 2.0569, 0.4258, 0.4543, -0.1609, -0.7749]	7.6112	2.7588	0
60	[-0.5264, -1.2569, -0.0190, 0.0343, -0.1756, -0.0075, -0.4907, -1.0359]	7.6517	2.7662	0
306	[1.8901, 1.3029, -0.3499, -0.6796, -0.0962, -0.9887, -0.4254, 1.2261]	10.1002	3.1781	1
90	[-0.8285, -1.3899, -1.4252, 0.0343, -0.1756, -1.9122, -0.6274, -1.0359]	12.0188	3.4668	0
231	[0.6819, 0.4053, 0.6427, 0.9861, 2.6044, 1.9984, -0.6868, 1.1391]	12.6348	3.5545	1
346	[-0.8285, 0.5715, -2.1696, -1.1555, -0.6522, -0.5269, 0.5492, -0.9489]	13.7388	3.7066	0
618	[1.5880, -0.3261, 0.8082, -0.5606, -0.1756, -0.5991, 2.4150, 1.4871]	14.1486	3.7615	1

340	[-0.8285, 0.2723, -0.1844, -1.8693, -0.4026, -0.9310, 0.0084, -0.9489]	15.7103	3.9636	0
294	[-1.1305, 1.3029, -1.8388, 0.0343, -0.1756, -1.5082, -0.6393, 2.7921]	15.8654	3.9831	0

- **Identify nearest neighbors and predicted class**

Identification of Nearest Neighbors

For K = 3, we select the top three samples from the ranking above:

1. Index 535 (Distance: 2.6366)
2. Index 718 (Distance: 2.7588)
3. Index 60 (Distance: 2.7662)

Predicted Class (Majority Vote)

The predicted class is determined by the most frequent label among the K neighbors:

- Neighbor 1 (Index 535): Outcome 1 (Diabetic)
- Neighbor 2 (Index 718): Outcome 0 (Non-diabetic)
- Neighbor 3 (Index 60): Outcome 0 (Non-diabetic)

Tally:

- Outcome 0: 2 votes
- Outcome 1: 1 vote

Predicted Class: 0 (Non-diabetic)

Part 4: Model Evaluation (20 pts)

Evaluate your model using:

• Accuracy

• Confusion Matrix

K Value	Accuracy	Confusion Matrix (TN,FP,FN,TP)
K=3	74.68%	[[81, 18], [21, 34]]
K=5	77.27%	[[85, 14], [21, 34]]
K=7	75.97%	[[84, 15], [22, 33]]

1. Which value of K performed best?
In this evaluation, K=5 performed the best with an accuracy of 77.27%. It achieved the highest number of correct classifications (119 out of 154 test samples) and showed a better balance in reducing False Positives compared to K=3.

2. Why does performance change with different K values?
Performance changes because the K value controls the "smoothness" of the decision boundary.

- A low K focuses on local patterns and is highly sensitive to individual data points.
- A high K considers a broader neighborhood, which smooths out the influence of outliers but may overlook smaller, valid clusters of data.

The performance shifts as the model tries to find the "sweet spot" where it captures enough local detail without being distracted by noise.

3. What happens when K is too small or too large?
When K is too small (e.g., K=1):
The model often suffers from Overfitting. It becomes overly sensitive to noise, outliers, and errors in the training data. The decision boundary becomes very complex and "wiggly" as it tries to accommodate every single point, which usually leads to poor performance on new, unseen data.
When K is too large (e.g., K=100):
The model suffers from Underfitting. The decision boundary becomes too simple or flat. If K is too high, the neighborhood becomes so large that it ignores local variations entirely, often defaulting to predicting the majority class of the entire dataset. This results in a model that is too "blunt" to make accurate specific predictions.

Part 5: Analysis and Reflection (20 pts)
Provide a discussion (minimum 300 words):

Strengths and Limitations of KNN
Working with the **K-Nearest Neighbors (KNN)** algorithm in this lab highlighted its greatest strength: its intuitive nature. As a "lazy learner," it doesn't spend time building a complex model upfront; instead, it looks at the data points closest to a new entry to make a decision. This simplicity makes it a great starting point for classification tasks, as it doesn't make rigid assumptions about how the data is distributed.

However, the "cost" of this simplicity became clear during implementation. Because KNN calculates the distance to every single point in the training set for every prediction, it can be computationally heavy and slow as the dataset grows. I also noticed how sensitive it is to the scale of the data. Without the manual Min-Max normalization I applied, a high **Glucose** reading would completely drown out smaller (but equally important) features like **BMI**, leading to skewed results.

When to Use KNN

Based on this experiment, KNN is most effective for smaller, well-organized datasets where "similarity" is a clear indicator of the outcome—such as in medical diagnosis where patients with similar vitals often share a diagnosis. It is less appropriate for massive, high-dimensional datasets where the distance calculations would become a bottleneck for performance.

Personal Observations and Experiment Results

During the lab, I observed that the choice of **K** is the most critical decision in the process. When I used a very small K (like K=1 or K=3), the model was too "twitchy," reacting to every outlier or bit of noise in the data. This is a classic case of **overfitting**. Conversely, increasing K too much (like K=15 or higher) made the model too general, causing it to ignore subtle patterns. Finding that "sweet spot"—usually a balanced K value—was key to getting the best accuracy.

A major takeaway from the preprocessing stage was the issue of "zero" values in columns like **Insulin** and **SkinThickness**. Since it's biologically impossible for a living patient to have a skin thickness of 0, these were clearly missing values. Cleaning this data and applying feature scaling weren't just extra steps; they were the foundation that allowed the KNN algorithm to actually function correctly. Ultimately, this lab showed me that while the algorithm itself is simple, the quality of the results depends entirely on how carefully you prepare your data.